

THE DEEP-STUDY TIER · CANYON NOTES

CANYON NOTES

CMPINF-2100 · Introduction to Data-Centric Computing

Part 1 - the first half of the semester, woven into one lesson

Modules 1–8 · Foundations · Python · Statistics · NumPy · pandas · Visualization

Verified under Double Diamond QC · every number run, not recalled

Authored and directed by **Victor S. (DatJavaClass)**, with assistance by Claude.

Shared with the CMPINF-2100 cohort. A study aid, not a textbook - check anything important against the course material before a quiz.



Codeman, resident mascot. He thinks 35 pages is showing restraint.

How to read these notes

A Canyon Note is the deepest tier of study note: where a Cliff Note is terse and a Cliff Jumper Note weaves one module together, a Canyon Note weaves the **whole half-semester** into a single connected lesson - deeper and layered, but never padded. It is meant to be read for long stretches, so it is built to relax the eye.

The course is not eight separate topics; it is one idea told six ways. A handful of threads run the length of it - the data table, the `[action for x in list]` idiom, the $1/\sqrt{n}$ shrink of uncertainty, "axis is the direction you collapse." Wherever a thread resurfaces, you will see a **Through-line** box. Follow those and the modules stop feeling like a list and start feeling like a story.

THE FIVE BOXES, AND WHAT EACH MEANS

KEY IDEA

The load-bearing fact for a section. If you remember one thing, remember this.

→ THROUGH-LINE

A thread that connects this idea to another module. The weave that makes the half-semester one lesson.

WORKED EXAMPLE

A concrete trace you can follow by hand or type into a cell. Every one here was run on a real machine.

⚠ TRAP

The specific way this breaks - an off-by-one, a silent wrong answer, a name that isn't defined. The traps are where the points live.

EXAM NOTE

A place where the course states something the loose way. Understand the precise version for real work, but **answer the course's way on a quiz**.

ONE NOTE ON VERSIONS

Every code result here was run on the standard Anaconda stack: **Python 3.13, NumPy 2.3, pandas 2.3, matplotlib 3.10, seaborn 0.13**. A few facts changed between pandas 2.x and the newer pandas 3.0 (text-column dtype, Copy-on-Write); where that matters, the note says what is true on your install and flags the change.

The weave: twelve through-lines

These are the threads to watch. Each one is introduced once and then pays off again and again. The whole point of a Canyon Note is to make them visible.

Thread	What it is	Where it runs
The data table	rows = observations, columns = attributes, cells = values	M1 → NumPy array (M5) → pandas DataFrame (M6)
Insight vs. prediction	patterns <i>across columns</i> vs. a value for a <i>new row</i>	M1 goal → pipeline step 5 (modeling)
The comprehension	[action for x in old] - transform a list in one line	M1-2 → variance (M4) → SEM sweep (M5)
Exclusive stop	a range/slice goes up to but <i>not including</i> the end	lists/strings (M2) → range → NumPy → pandas
** is power	exponent; ** 0.5 is a square root	M2 → SD = variance ** 0.5 (M4)
The uncertainty chain	variance(n-1) → SD → SEM = s/√n → CI = $\bar{x} \pm 2 \cdot \text{SEM}$	M4 formula → <i>simulated</i> in M5 & M6
The 1/√n law	quadruple the sample, halve the error	M4 → NumPy sim (M5) → pandas sim (M6)
Axis = collapse	“axis” is the direction an operation crushes the grid	NumPy (M5) → .mean(axis=1), groupby (M6-7)
Mask, then apply	build booleans with & ~ in parens, then index	NumPy filter (M5) → pandas .loc (M6)
Views vs. copies	a slice can point at the same memory	NumPy views (M5) → pandas Copy-on-Write (M6)
ddof=1	the sample statistic divides by n-1	M4 idea → NumPy → pandas default (M7)
EDA → pictures	explore numerically, then see the distribution	missing-values/describe (M7) → plots (M8)

Contents

1. **Part I - Foundations & the Pipeline** what data is, the table, the two goals, the six steps
2. **Part II - Programming in Python** data types, slicing, loops, errors, the comprehension
3. **Part III - Quantifying Uncertainty** distributions, variance, SEM, the CLT, confidence intervals

4. **Part IV - NumPy, the Numeric Engine** arrays, RC, axis, reshape, views, random, the SEM simulation
5. **Part V - pandas, Labeled Data** Series, DataFrames, filtering, NaN, group-by
6. **Part VI - Visualization** matplotlib, the plot-choice rule, bars, histograms, boxplots, heatmaps

Closes with **The Weave** (a one-page synthesis) and a per-part confidence table.

Foundations & the Pipeline

The one big idea: data science reads patterns across the columns of a table (insight) and predicts the values of a new row (prediction) - and a six-step pipeline turns those two goals into a repeatable process. This part plants two through-lines: the data table, and insight-vs-prediction.

1. What the course is really teaching

CMPINF-2100, *Introduction to Data-Centric Computing*, advertises three skills - programming for data analysis in Python, visualization through code, and making predictions from patterns. But the framing underneath them matters more: this is a **practice**, a way of working with data through programming, not a checklist of tricks. It assumes no programming background, and everything later in the program builds on it.

2. Data is structured - and never “raw truth”

Data is structured information about the world: observations and recorded measurements. Some of it is numeric outright; some is pulled from text, images, or audio and processed into structured form first. The caveat matters more than the definition:

KEY IDEA

Data is not raw truth. Somebody - or a sensor someone designed - did the observing, so data can be incomplete, imperfect, and subjective. Keep that human/instrument layer in view; it is the thread that reappears at every later stage of the pipeline.

And data has a **shape**: a table.

	name	material	year built	
row →	Parthenon	Marble	438 BCE	one observation
row →	Colosseum	Travertine	80 CE	one observation
each column is one attribute · each cell is that attribute's value				

Every dataset in this course is a table. Even here the provenance caveat bites: does “year built” mean started or finished, and how precise is an estimate for an ancient ruin? Interrogating what an attribute means is part of the work, not a preliminary to it.

→ THROUGH-LINE: THE TABLE

Hold this row/column/cell picture. NumPy (Part IV) keeps the numeric core of it; pandas (Part V) keeps the whole thing, labels and all. The definition of data science is about to be read straight off this table.

3. Data science: two goals, read off the table

Data science is collecting, processing, and manipulating data toward two goals, and both map cleanly onto the table:

- **Insight** - finding patterns and relationships *among attributes*, i.e. **across the columns**. Across a set of ancient buildings: which materials appear in which eras?
- **Prediction** - estimating attribute values for observations *not yet seen*, i.e. **a new row**. A newly found ruin: given its material and location, predict its likely build year from the patterns in known ruins.

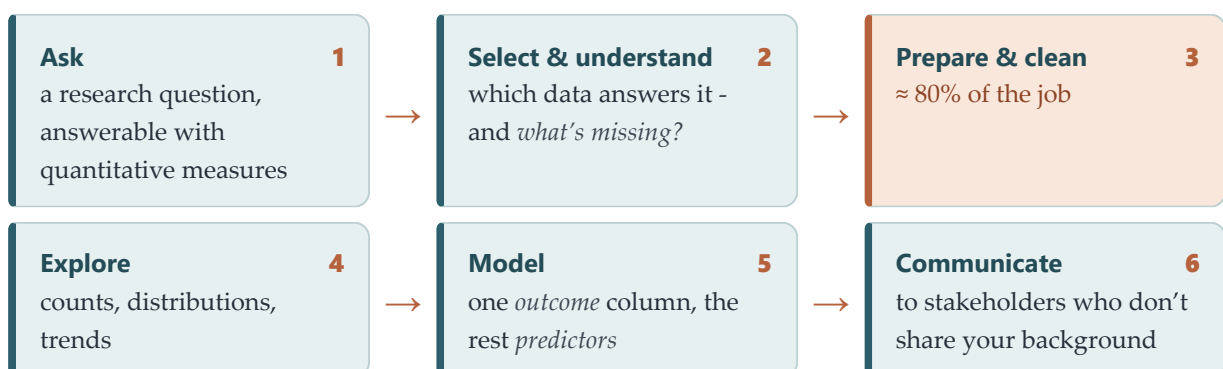
→ THROUGH-LINE: INSIGHT VS. PREDICTION

“Across the columns” and “a new row” are not throwaway phrases - they become *step 5* of the pipeline below, where one column is the **outcome** you predict and the others are the **predictors**.

Where it sits among neighbors. *AI* also predicts from data, but its goal is building systems that accomplish tasks (generate language or images); for data science, the insight or prediction *is* the product. *Machine learning* shares the learn-then-predict technique but is more computational. *Statistics* is the closest relative - used *inside* data science for pattern-finding - but trends more abstract and cares less about cleaning and computation. *Database engineering* handles storage. Mental model: data science sits at the **intersection** of database-style organization and ML-style prediction.

4. The pipeline: the backbone of everything

The two goals say *what* you produce; the **data science pipeline** says *how* you get there. It rests on **CRISP-DM** (Cross-Industry Standard Process for Data Mining), and it teaches one durable habit: when you are buried in technical detail, **pop up a level** and name which step you are on.



Steps 3 and 4 are iterative, not sequential: clean → explore → spot an outlier → back to cleaning. An outlier may be a mistake to drop or the real signal - judgment required. Step 5 is the “prediction” goal made operational; step 6 closes the loop back to step 1.

KEY IDEA

The center of gravity is **step 3**: roughly **80%** of real work is preparing and cleaning - handling missing values, dropping irrelevant fields, fixing granularity, combining sources, and re-typing data stored in the wrong form. The rest of this course is largely tools for that 80%.

PART I RECAP

Data is structured, human-mediated information shaped like a **table**. Data science reads patterns **across columns** (insight) and predicts **unseen rows** (prediction). The **CRISP-DM six-step pipeline** turns those goals into a repeatable process whose center of gravity is the **~80% spent cleaning**. Hold the table and the two goals; everything downstream is built on them.

Programming in Python

*The one big idea: **programming is adapting examples, one small step at a time** - and the early discomfort is normal. This part is the keyboard half of the foundation: the data you hold (types, lists, slicing), the ways you repeat work (loops), the ways code breaks (errors), and the single most-used data idiom, the list comprehension.*

5. The values you hold: data types

A **variable** is a name you assign with `=`: `x = 3` puts the value on the right into the name on the left. Python is **case-sensitive** (`fruit` and `Fruit` are different names). Every value has a **type**, and the type decides what you can do with it.

WORKED EXAMPLE - THE FIVE EARLY TYPES

```
type(3)           # <class 'int'>
type(0.5)         # <class 'float'>
type("hello")    # <class 'str'>
type(True)       # <class 'bool'>
type([1,2,3])    # <class 'list'>

3 + 0.5          # 3.5 → mixing an int and a float gives a float
```

A **list** is an ordered, **mutable**, **heterogeneous**, zero-indexed container - it can even hold mixed types and nested lists, like `["apple", 42, 3.14, True, [1,2,3]]` - and `len(list)` gives its element count.

6. Reaching into a list: indexing & slicing

Positions start at 0, and negative numbers count from the end. Slicing is `list[start:stop:step]`, and the rule that will save you a hundred bugs:

KEY IDEA - EXCLUSIVE STOP

A slice is **inclusive of start** but **exclusive of stop**: it goes up to, but does not include, the end index. `flowers[0:2]` gives you indices 0 and 1 - two items, not three.

WORKED EXAMPLE - SLICING, RUN FOR REAL

```
flowers = ["rose", "lily", "tulip", "jasmine"]
flowers[0]      # 'rose'    (first)
flowers[-1]     # 'jasmine' (last, counting from the end)
flowers[0:2]    # ['rose', 'lily'] (stop 2 excluded)
flowers[1:]     # ['lily', 'tulip', 'jasmine']
flowers[: -1]   # ['rose', 'lily', 'tulip'] (all but last)

nums = [0,1,2,3,4,5,6,7]
nums[1::2]      # [1, 3, 5, 7] (every 2nd from index 1)
nums[::-1]      # [7,6,5,4,3,2,1,0] (reversed, step -1)

"Python"[1:4]   # 'yth' (strings slice like lists of characters)
```

→ THROUGH-LINE: EXCLUSIVE STOP

This exact rule governs `range` (next), then NumPy's array slices in Part IV, then pandas's positional indexing in Part V. Learn it once here and it pays off four times.

7. Repeating work: the `for` loop and `range`

A Python `for` loop is a **for-each** loop: it hands you one item at a time and stops on its own when the collection runs out. It is not a counter, and - because the collection is finite - **it cannot run forever**.

WORKED EXAMPLE

```
for f in ["apple", "orange", "banana"]:
    print(f)
# apple / orange / banana, one per line
```

When you need numbers instead of items - to do something a set number of times, or to get position indices - reach for `range`, which manufactures a sequence of integers. It has the same exclusive-stop rule as slicing.

WORKED EXAMPLE - RANGE'S THREE DIALS

```
list(range(5))      # [0, 1, 2, 3, 4]    five numbers, last is 4
list(range(2, 5))   # [2, 3, 4]         start, stop
list(range(0, 10, 2)) # [0, 2, 4, 6, 8]  start, stop, step
list(range(0, 21, 2)) # [0,2,...,20] push the stop past 20 to include it

print(range(0, 10, 2))# range(0, 10, 2) - Lazy: prints the recipe, not the numbers
```

`len()` returns how many items are in something, and `range(len(list))` produces exactly the valid indices `0 ... n-1` - useful when you genuinely need the position number. If you only want

the items, `for item in list:` is cleaner.

⚠ TRAP - `RANGE(A_LIST)` IS NOT A THING

`range` needs a **number**, not a list. `range(masters)` crashes; it must be `range(len(masters))`.

NESTED LOOPS: THE ODOMETER

Put a loop inside a loop and the inner one runs *completely* for every single pass of the outer one - like a car odometer, where the right wheel spins a full turn before the left wheel clicks once.

⚠ TRAP - NESTING MULTIPLIES

Total iterations are outer × inner, not plus. Two loops of 100 is 10,000 passes; three nested loops of 100 is a **million**. Know your multiplication before you hit Run. (This is exactly how you will later walk the rows × columns of a 2-D array.)

8. The `while` loop, and the loop Python pretends not to have

A `while` loop repeats as long as a condition stays true. Unlike `for`, it has no collection to exhaust, so **you** are responsible for making the condition eventually false. It has three required parts.

i	i < 5 ?	action	i after
0	True	print 0	1
1	True	print 1	2
2	True	print 2	3
3	True	print 3	4
4	True	print 4	5
5	False	stop	-

Trace: `i = 0` (initialise) · `while i < 5:` (condition) · `print(i); i += 1` (advance)

⚠ TRAP - THE INFINITE LOOP

Forget the advance step (`i += 1`) and `i` stays 0 forever, `0 < 5` is always true, and the loop never ends. In Jupyter this freezes the cell - know where the square **stop** button is before you write your first `while`.

for vs. while: use `for` when you know the count or are walking a known group (it cannot infinite-loop); use `while` when the number of repetitions is unknown and condition-driven (until a user quits, a value converges, input is valid). Python has **no do-while** keyword; you fake “run once, then test” with `while True:` + a bottom `if ...: break`, or by priming a sentinel variable.

STEERING A LOOP: BREAK AND CONTINUE

`break` leaves the loop entirely; `continue` skips the rest of *this* pass and moves to the next. Same setup, one word apart:

WORKED EXAMPLE - ONE WORD CHANGES THE OUTPUT

```
for n in [1,2,3,4,5]:
    if n == 3: break      # prints 1, 2 then stops entirely
    print(n)

for n in [1,2,3,4,5]:
    if n == 3: continue  # prints 1, 2, 4, 5 (3 skipped, loop continues)
    print(n)
```

⚠ TRAP - ORDER IS LOGIC, NOT DECORATION

When `continue` and `break` share a body they are checked top to bottom. With `for n in [1..8]: if n%2: continue; if n>6: break; print(n)` the output is 2, 4, 6 - and the loop ends at 8, not 7, because 7 is odd and hits `continue` before the `break` is ever evaluated. (In a `while` loop, a `continue` placed *above* the `i += 1` jumps over it and strands the counter - an infinite loop.)

9. When code fails: try / except and the error taxonomy

Some operations fail at runtime - dividing by zero, turning 'cat' into a number, reading a missing file. `try / except` attempts the risky code and handles the failure instead of crashing (Python spells "catch" as `except` and "throw" as `raise`). The everyday data pattern is `try + continue`: process each row, skip the broken ones, keep going.

⚠ TRAP - THE BARE EXCEPT:

Writing `except:` with no type catches *every* error - including your own typos and logic bugs. The program stops crashing but starts producing silently wrong answers, which is worse. Always name the error you expect (`except ValueError:`) and let the unexpected ones crash loudly. A loud crash is information; a swallowed one is a landmine.

That trap points at the deeper Module-3 idea: there are two kinds of errors, and telling them apart is half of debugging.

Error	What happens	Example
Syntax error	malformed code - it won't run at all	<code>for n in [1,2,3]</code> with no colon
Logic error	runs fine, produces the wrong result - hides until you test	<code>return a + b / 2</code> when you meant <code>(a + b) / 2</code>

EXAM NOTE - "UNDEFINED VARIABLE"

The course lists *using an undefined variable* as a **syntax** error. In Python it is really a **runtime** error (a `NameError`): a true syntax error is caught *before* the program runs, but an undefined name is not caught until execution reaches that line, so the program has already started. Know the real distinction for debugging - but if a quiz keys to the course's wording, **answer the course's way**. (This exact bug returns in Part III.)

10. The one idiom this all builds toward: the list comprehension

Most data-prep work has the same shape: **take a list, do something to each element, collect the results into a new list**. The long form is a `for` loop with a collector you must create *before* the loop:

WORKED EXAMPLE - THE FOUR PIECES, THEN THE ONE-LINER

```
fruits = ["apple", "orange", "banana"]
salads = []                                # the collector, BEFORE the loop
for fruit in fruits:                       # fruit = the iterating variable (you name it)
    salads.append(fruit + " salad")        # the action on each element
# salads == ['apple salad', 'orange salad', 'banana salad']

# the same four pieces, collapsed to one line:
salads = [fruit + " salad" for fruit in fruits]
#           [- action -/ for [var] in [old list]
```

The brackets do two jobs the loop needed separate lines for: they **create** the new list and **collect** each result - which is why a comprehension needs no `salads = []` in front of it. The same grammar works with numbers, and that is where the module's one new operator shows up:

WORKED EXAMPLE - SQUARING, AND THE ****** OPERATOR

```
[n ** 2 for n in [1, 2, 3, 4]]    # [1, 4, 9, 16]

3 ** 2    # 9    exponentiation
9 ** 0.5  # 3.0  a square root is just the 0.5 power
3 * 2     # 6    multiplication, NOT a power
3 ^ 2     # 1    bitwise XOR, NOT a power (0b11 XOR 0b10 = 0b01)
```

⚠ TRAP - **^** IS NOT "TO THE POWER OF"

In ordinary math notation **^** often means a power. In Python it is **bitwise XOR**: `3 ^ 2` is `1`, not `9`. The power operator is ******.

→ THROUGH-LINE: **** AND THE COMPREHENSION**

Both return immediately: in Part III the standard-deviation function is literally `variance ** 0.5`, and the variance itself is a comprehension over squared deviations. You are not learning these for their own sake - they are the building blocks of the statistics code.

One framing worth keeping: a comprehension is **syntax, not a method**. Methods like `.append()` are functions you call with parentheses on an object; syntax - `for`, `if`, comprehensions - is grammar you write *in shape*. There is no “comprehension function” to import. (The same grammar makes `{k:v for ...}` a dict and `{x for ...}` a set.)

11. How to actually program (Module 3)

If you are new, the early difficulty is not really Python - it is the shift from **point-and-click** (a fixed menu of choices) to **writing the commands yourself**. The lecture’s word for the feeling is **unmoored**, and it is normal and near-universal, usually settling within weeks. The remedy is a process and a mindset.

KEY IDEA - WORKING BEATS ELEGANT

There is no single “best way,” so **don’t optimize early**. Your first goal is just a solution that **works**; elegance and efficiency are later concerns, earned only after something runs.

The process is a loop: **Design** → **Write** → **Correct Errors** → **Test**, looping back as needed. Design means: think input and output first; write a concrete example (reuse it to test); **break the problem into steps**, starting from the step you already know (often the first, and often the last); then write pseudocode. And the truth that takes the pressure off:

KEY IDEA

Almost all programming is adapting examples. You will rarely find an exact match for what you need; adapting one is itself the core skill, not cheating. In this course, your examples come mostly from the course material. The minimum bar even when you use AI is being able to **understand and correct** the output, not just paste it - you are here to learn, and interviews test the code *you* can produce.

PART II RECAP

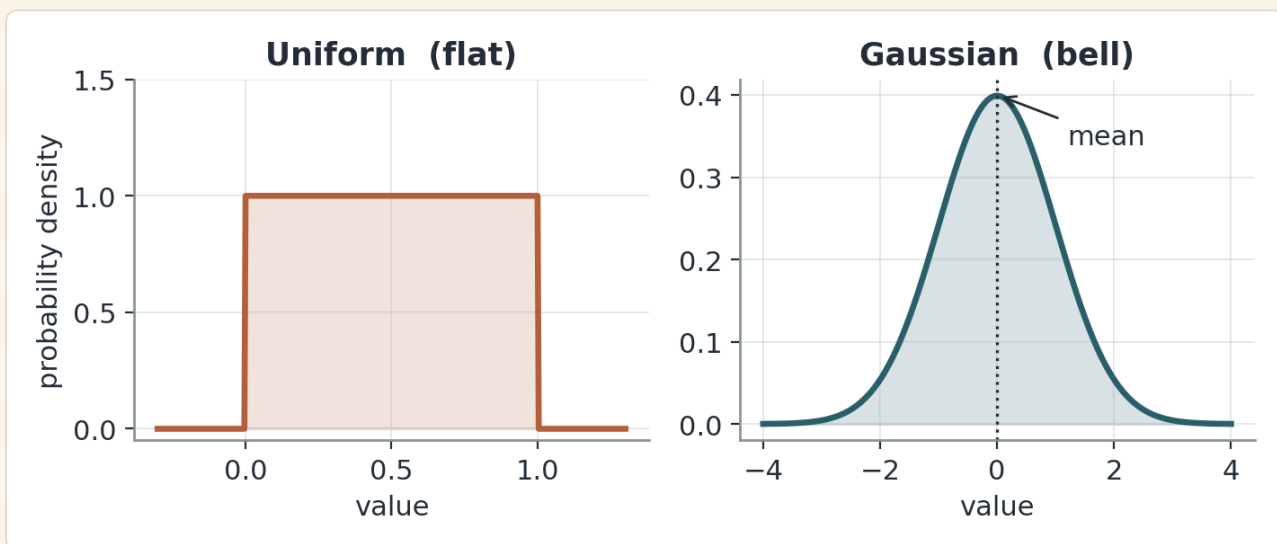
Values have **types**; lists are ordered and zero-indexed; slices and `range` share the **exclusive-stop** rule. `for` walks a known group (safe), `while` runs until a condition flips (you must make it stop); `break` exits, `continue` skips, and order matters. Errors split into **syntax** (won’t run) and **logic** (runs but wrong). The **list comprehension** `[action for x in old]` is the one-line workhorse of data prep, and `**` is its power operator (with `** 0.5` a square root). Above all: get something that *works*, one small step at a time, by adapting examples.

Quantifying Uncertainty

*The one big idea, and it is a single escalating question: **when you compute an average from a sample, how much can you trust it?** Random sampling gives the raw material; variance and SD measure the spread of the data; the standard error measures the spread of the average; and a confidence interval turns that into an honest range. Read the whole module as that one chain.*

12. Randomness means sampling from a distribution

In this course, “random” usually means sampling from a probability distribution - not an unstructured number. A distribution maps each possible outcome to its probability. Two matter here, and they look like this:



Uniform (flat): every value equally likely; `random.random()` draws this on the range the course writes as `[0, 1]`.

Gaussian = normal = bell curve (three names, one thing): a mean at the center where draws cluster, tails that taper but never quite reach zero.

You generate a single Gaussian value with `random.gauss(mu, sigma)`, and the two arguments are independent dials: **mu** slides the bell (the mean), **sigma** sets its width (the standard deviation) and is the only dial that touches shape. A single draw is just a number - the bell only appears in the *collection*, which is why you draw a thousand and histogram them.

KEY IDEA - THE SEED PINS THE SEQUENCE

Pseudo-random generation walks a fixed pre-computed sequence; the **seed is just where you start**. So `random.seed(2100)` followed by `random.random()` always gives `0.7640211...`, and the next call always gives `0.4692037...` - each call advances one step. Reproducibility is the point: it makes your screen match the lecture and a simulation gradeable.

⚠ TRAP - RESEEDING REWINDS

Seed *once*, then loop. Re-seeding before *every* draw sends you back to position one, so you get the same value over and over.

13. Variance and standard deviation: the spread of the data

Variance and SD measure how dispersed the data are around the mean - for *any* distribution, not just a bell. The recipe is mechanical, and it reuses Part II's comprehension and `** 0.5` :

WORKED EXAMPLE - THE VARIANCE TRACE

Variance of `[48, 49, 50, 51, 52]` , whose mean is 50:

value	value - mean	squared
48	-2	4
49	-1	1
50	0	0
51	1	1
52	2	4
sum of squares		10

variance = $10 / (5 - 1) = 2.5$, and SD = $\sqrt{2.5} \approx 1.58$. Compare `[0, 25, 50, 75, 100]` : same mean 50, but variance **1562.5** and SD ≈ 39.5 . *Same mean does not mean same spread.*

```
def my_variance(x):
    xbar = sum(x) / len(x)
    squared = [(xn - xbar) ** 2 for xn in x]    # ← a comprehension, again
    return sum(squared) / (len(x) - 1)        # ← divide by n-1 (the -1 inside)

def my_sd(x):
    return my_variance(x) ** 0.5              # ← ** 0.5 IS the square root
```

⚠ TRAP - THE MISSING PARENTHESES

`sum(sq) / (len(x) - 1)` gives 2.5; `sum(sq) / len(x) - 1` gives **1.0**, because Python divides before it subtracts. The `-1` must live *inside* the denominator.

→ THROUGH-LINE: WHY N-1, AND DDof

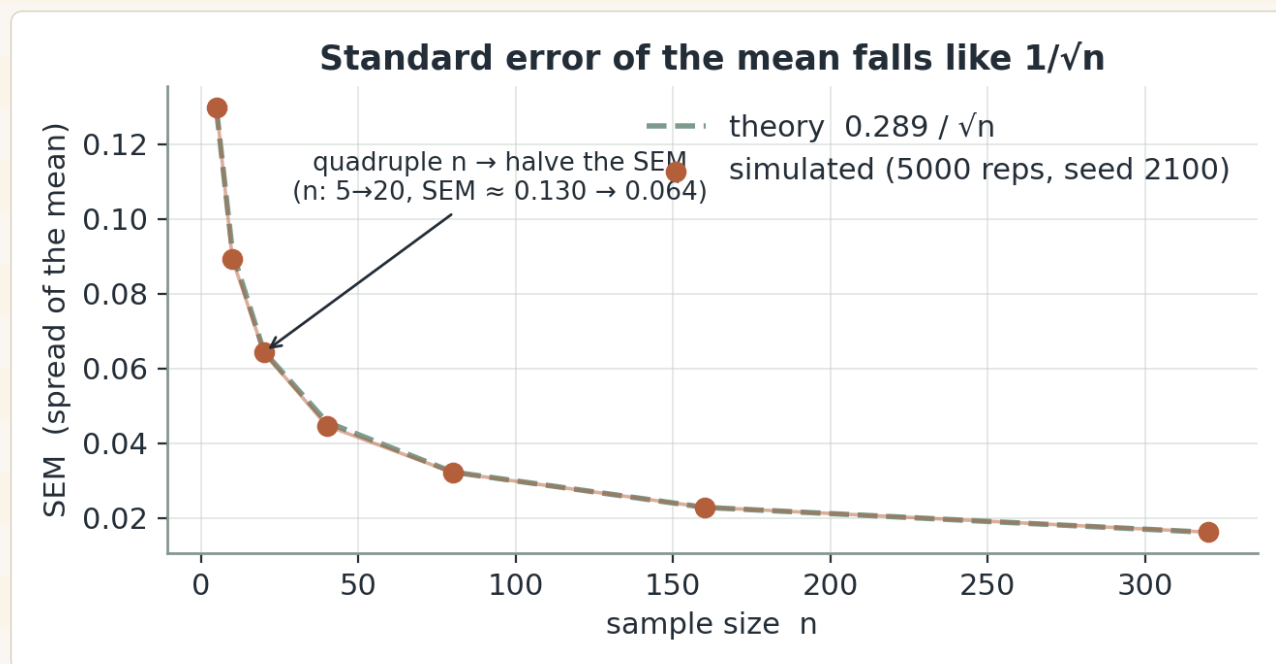
Dividing by `n` systematically under-estimates a sample's spread; the fix (Bessel's correction) is the **degree of freedom** you spend pinning the mean. NumPy spells it `ddof` - read it as "denominator = `n - ddof`." `ddof=1` is the sample statistic (this course); `ddof=0` is the population. The same `ddof=1` returns in Part IV (NumPy) and is pandas's default in Part V.

Libraries disagree on the default. For that same list (true sample variance 2.5): `statistics.variance` → 2.5 out of the box; `numpy.var` → 2.0 (population) until you pass `ddof=1`; `pandas.var()` → 2.5 (sample) by default. When in doubt, $n-1$.

14. The standard error of the mean: the spread of the *average*

Here is the pivot most people miss. SD describes how scattered the **individual data points** are. The **standard error of the mean (SEM)** describes how much the **sample average itself** would bounce around if you drew many samples of size n . The formula is $SEM = s / \sqrt{n}$, and two things fall straight out of that $\sqrt{n}$:

- SEM is always **smaller** than SD (for any real sample) - an average is steadier than its values.
- Because the term is $1/\sqrt{n}$, **quadrupling n halves the SEM**.



Simulated SEM (terracotta dots, 5000 replications, seed 2100) sitting exactly on the theory curve $0.289 / \sqrt{n}$. Quadruple the sample (5→20) and the SEM halves ($\approx 0.130 \rightarrow 0.064$). That square root is why “just collect more data” gets expensive fast.

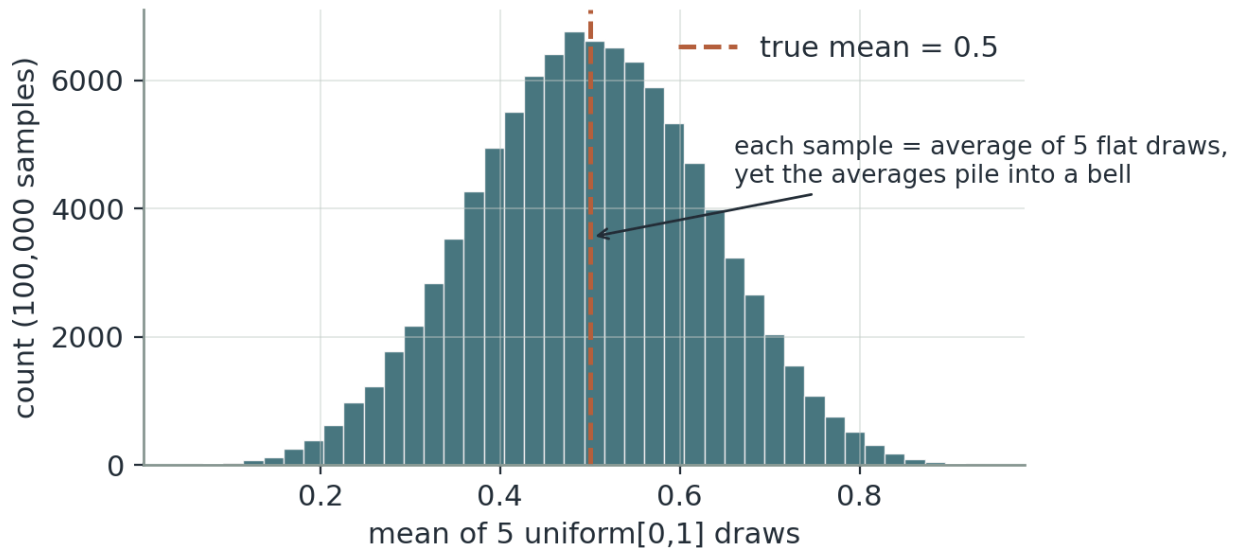
⚠ TRAP - SD IS NOT SEM

SD is the spread of the data points (“how scattered are my values?”). SEM is the spread of the mean (“how much can I trust my average?”). They answer different questions; do not conflate them.

15. The Central Limit Theorem: why a mean is trustworthy at all

Why can we put an error bar on a mean? Because of the **Central Limit Theorem**: the distribution of the sample *mean* is approximately normal **regardless of the shape of the underlying data**, and the approximation sharpens as n grows. The cleanest way to see it is to build it from the flattest possible source:

Central Limit Theorem: a flat source makes a bell of means



*Each sample is the average of five flat uniform draws - yet the averages pile into a **bell**, peaked at the true mean 0.5. Intuition: there are many ways for five draws to average near 0.5, but for them to average near 0 they must all be tiny at once (rare). This bell is the hinge the rest of the module turns on.*

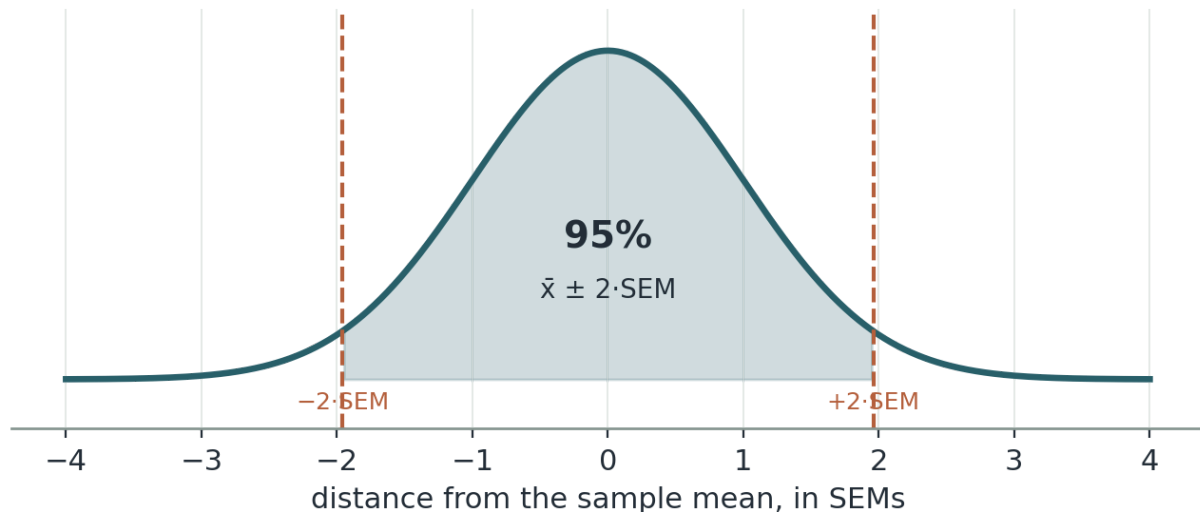
16. Confidence intervals: stating the uncertainty

A **confidence interval** is the range you report for where the true mean plausibly sits - the “margin of error” from polls. Because sample means are normal, about 95% of them land within ~2 standard errors of the center, which gives the course’s formula:

KEY IDEA

$CI_{95\%} = (\bar{x} - 2 \cdot SEM, \bar{x} + 2 \cdot SEM)$. The “2” is the rounded 2-sigma rule; the more exact 95% multiplier is **1.96**, but the course uses 2. SEM is the *width* of the interval: big SEM → wide CI (“could be anywhere”); small SEM → narrow CI (“right around here”).

A 95% confidence interval = the central 95% of the bell of means



The 95% interval is the central 95% of the bell of means. As n grows the bell narrows, so the interval tightens: at $n=5$ it is far too wide to pin down the true 0.5; at $n=1000$ it brackets 0.5 closely. Larger $n \rightarrow$ smaller SEM $\rightarrow$ narrower CI $\rightarrow$ more confidence.

EXAM NOTE - WHAT "95% CONFIDENT" MEANS

It is tempting (and the course phrases it this way) to say "there is a 95% chance the true mean is in *this* interval." Strictly, that is the one thing a confidence interval does **not** say: the true mean is a fixed number and the interval is the random thing, so this particular interval either contains it or it doesn't. "95% confident" is a statement about the **procedure** - repeat the sampling many times and about 95% of the intervals you build will contain the true mean. Carry the precise version into real work; **answer the course's way on a quiz.**

A small- n caveat: the $\pm 2 \cdot \text{SEM}$ rule leans on the large- n normal approximation. At very small n (like 5) the interval actually captures the true mean a bit *less* than 95% of the time (closer to ~87% in simulation), because the sample SD is itself noisy. Treat a small- n interval as a rough bracket; proper small-sample work uses a wider, t-based multiplier.

→ THROUGH-LINE: THE BUG RETURNS

The lecture builds this as a stack of functions - `my_average` $\rightarrow$ `my_variance` $\rightarrow$ `my_standard_deviation` $\rightarrow$ SEM $\rightarrow$ CI - and deliberately shows a bug: writing a bare `variance` `** 0.5` references a name that was never defined, raising a `NameError` at runtime. That is exactly Part II's syntax-vs-runtime point: the program *starts*, then fails on that line. Make SD call `my_variance(x)`.

PART III RECAP

“Random” means drawing from a distribution (uniform = flat, Gaussian = bell). **Variance** = mean of squared deviations over $n-1$, and **SD** = $\sqrt{\text{variance}}$, measure the spread of the *data*. The **SEM** = $s/\sqrt{n}$ measures the spread of the *mean* and shrinks like $1/\sqrt{n}$. The **Central Limit Theorem** makes the bell of means normal whatever the source, so a **95% CI** $\approx \bar{x} \pm 2 \cdot \text{SEM}$ states the uncertainty - correctly read as “95% of such intervals would capture the true mean.”

NumPy, the Numeric Engine

The one big idea: NumPy holds the numeric core of the table as an `ndarray`, and one fact - rows-first, columns-second, with “axis” = the direction you collapse - governs nearly everything. Learn that spine and the rest hangs off it. The part ends by simulating Part III’s standard error.

17. Why NumPy, and the two anchors

Data lives as a table; NumPy strips the headers and holds just the **numeric core** as an `ndarray` (n-dimensional array), which is what makes the math fast. Two attributes anchor everything (both nouns, no parentheses):

- `ndim` - the number of dimensions (1-D = a list of numbers; 2-D = a table).
- `shape` - (rows, columns), and the axes match that order: **axis 0 = rows (down), axis 1 = columns (across)**.

KEY IDEA - RC

Shape is **(Rows, Columns)** and indexing is `arr[row, column]` - one bracket, one comma, with `:` meaning “all of that axis.” This RC habit is the single most reused idea in the module. (And it inherits the same exclusive-stop slicing from Part II.)

Build 1-D arrays with `np.arange(6)` → `[0 1 2 3 4 5]` (stop-exclusive, like `range`), `np.array([...])`, `np.zeros(5)`, `np.ones(5)`, or `np.linspace(0,1,50)` (50 points, endpoint *inclusive*). Build 2-D from a list of lists, or stack 1-D arrays with `np.vstack([a,b])`. Pass shapes as a *tuple* - `np.zeros((2,2))`, note the double parentheses.

⚠ TRAP - ARRAYS DO MATH, LISTS CONCATENATE

`[1,2] + [1,2]` joins two lists into `[1,2,1,2]`; `np.array([1,2]) + np.array([1,2])` adds element-wise into `[2, 4]`. And `==` on arrays returns a **boolean array**, not one True/False - use `np.array_equal(a, b)` for a single whole-array answer.

18. Reshape vs. transpose: re-fence, or fold?

Both change an array’s `shape` without changing its `size`, but they are different tools, and conflating them is the module’s number-one confusion.

start (4×3)
reading order 0...11

0	1	2
3	4	5
6	7	8
9	10	11

.reshape(2,6)
order kept

0	1	2	3	4	5
6	7	8	9	10	11

.T transpose (3×4)
rows fold into columns

0	3	6	9
1	4	7	10
2	5	8	11

*Reshape keeps the reading order and just redraws the fences (rows×cols must equal `.size` ; pass `-1` for one dimension and NumPy solves it). **Transpose** (`.T` , an attribute - no parentheses) swaps every coordinate `[r,c]→[c,r]` , so a 4×3 becomes a 3×4. It is a fold across the diagonal, not a spin. Both are reads - they return a new array, so capture the result.*

Flattening back to 1-D: `.ravel()` always returns a flat array (but as a *view* - see below); `.squeeze()` only removes size-1 dimensions, so it is *not* a general flattener (`.squeeze()` on a 3×4 returns it unchanged). For a guaranteed flat result use `.ravel()` .

19. Views vs. copies: the side-effect trap

⚠ TRAP - A SLICE IS A VIEW

A plain slice is a **window onto the same memory**. Mutating the slice mutates the original:

```
sub = arr[0:2, 0:2] # a VIEW
sub[:] = 999       # this ALSO changes arr

sub = arr[0:2, 0:2].copy() # an independent box
sub[:] = 999             # arr stays untouched
```

The mental model: a view is your hand resting on part of the same box (poke it, the original changes); a copy is a second box built independently. Ask: *did a second box get built* (`.copy()`), or *am I holding part of the only box I had* (a plain slice)?

→ THROUGH-LINE: VIEWS → COPY-ON-WRITE

This exact hazard is why early pandas warned about editing a filtered subset, and why pandas 3.0 makes Copy-on-Write mandatory. We meet it again in Part V.

20. Filtering, and the axis that is the spine

Filtering is two stages: build a **boolean mask**, then **apply** it. Where the mask sits decides what gets filtered, and the `:` always sits in the axis you are *not* filtering: `arr[arr<5]` returns a flat array of matching elements; `arr[arr[:,0]>5, :]` keeps whole *rows* where column 0 > 5.

And then the one genuinely counter-intuitive idea in the module - the payoff of the RC spine:

1	2	3
4	5	6

↓ **axis=0** collapse the rows
 → one mean **per column**
`[2.5, 3.5, 4.5]`

1	2	3
4	5	6

→ **axis=1** collapse the columns
 → one mean **per row**
`[2.0, 5.0]`

For a reduction like `.mean()`, *axis* is not a selector - it is the direction the calculation crushes the grid, and the result keeps the axis you did not collapse.

KEY IDEA - "THE MEAN OF EACH ROW" IS `axis=1`

To get a per-row answer you sweep *across the columns*, which is `axis=1`. Safety check: a 2-row array should give you 2 row-means → that's `axis=1` (`axis=0` would give one value per column instead). `.std`, `.sum`, and friends all take the same `axis`.

→ THROUGH-LINE: `axis = collapse`

This is the rule you will reuse in Part V for `df.mean(axis=1)` and for the way a group-by reduces - same idea, one library up.

21. Random generation, and the SEM simulation that ties it together

Build one **generator object** and reuse it: `rg = np.random.default_rng(seed=2100)`. Then `rg.integers(low, high, size=...)` (with `high` exclusive), `rg.random(shape)` for floats in `[0, 1)`, and `rg.normal(mean, std, shape)`.

⚠ TRAP - THE GENERATOR IS STATEFUL

It advances on every draw, so one generator drawn twice gives *different* results - a seed does not freeze the output to one value, it fixes the *start of a sequence*. To get matching draws you need **two separate generators, seeded the same**, each on its first draw. (And `rg.integers(50, 100)` gives 50–99, not 50–100 - the high end is exclusive.)

The capstone fuses the whole module and reconnects to Part III. Read it as a stack of through-lines:

```
rg = np.random.default_rng(2100)
sem = rg.random((5000, 7)).mean(axis=1).std(ddof=1) # ≈ 0.1096
#           [ 2-D array ] [axis=1: per-row] [ddof=1: n-1]
```

Every piece is a callback: a **2-D array** shaped (replications, sample size); **axis=1** to take a mean per sample (§20); **ddof=1** for the unbiased SD (Part III's `n-1`); and the seeded **generator**. The result `≈ 0.1096` matches the analytic SEM of a uniform sample at `n=7` (`0.289/√7`). Sweep `n` with a list comprehension and you reproduce the `1/√n` curve from Part III - the formula, now produced by simulation.

PART IV RECAP

NumPy holds the numeric table as an `ndarray` with anchors `ndim` and `shape = (rows, cols)`, indexed **RC**. **Reshape** re-fences in reading order; `.T` folds a 4×3 into a 3×4. Slices and `.ravel()` are **views** (use `.copy()` to be safe). Filter by building a boolean mask and applying it; for stats, **axis is the direction you collapse**, so per-row means use `axis=1`. Build randomness from one stateful, seeded `default_rng` - and the capstone fuses a 2-D array, `axis=1`, and `ddof=1` to show the SEM shrinking like $1/\sqrt{n}$.

pandas, Labeled Data

*The one big idea: **pandas is NumPy's numeric core with labels bolted on.** A **Series** is values + an index; a **DataFrame** is several aligned Series sharing one index; and almost every operation is “reach in through the labels and select, add, filter, or group.” Two Part-IV threads come straight back: axis-as-collapse, and views-vs-copies.*

22. Series and DataFrame: the labeled table

A **Series** is one-dimensional - values plus an index - with two attributes: `.index` (the labels, default `RangeIndex 0...n-1`) and `.values` (the data, as a NumPy array). A **DataFrame** is the whole table: **columns are variables, rows are observations**, and it is effectively several aligned Series sharing one index.

⚠ TRAP - A SERIES INDEXES BY LABEL

This is the opposite of lists and NumPy: **negative indexing does not work**. With a default integer index, `s[-1]` looks for a *label* called `-1`, doesn't find one, and raises `KeyError`. To get the last element by *position*, use `.iloc[-1]`.

Build a frame with `pd.DataFrame(...)` from a list of lists, a NumPy array, or a dict (keys become column names). Inspect with `df.shape` (an attribute → `(rows, cols)`), `df.dtypes`, and `df.info()` (a method → non-null counts + memory).

EXAM NOTE - A TEXT COLUMN'S DTYPE

On your install (pandas 2.x) a text column's dtype is `object` - this is the course's “quiz fact,” so **answer `object`**. (On the newer pandas 3.0 it prints the dedicated `str` dtype; numeric columns stay `int64` / `float64` either way.)

23. Columns, and the single-vs-double bracket rule

Most data work is column work. Select with brackets (always safe) or dot notation (fails on spaces or names that collide with methods). The shape of what comes back depends on the brackets:

KEY IDEA - ONE BRACKET VS. TWO

`df["wins"]` → a **Series** (1-D). `df[["city", "wins"]]` → a **DataFrame** (2-D). The double brackets are “select” wrapped around a *list* of names. Single column → Series; a list of columns → DataFrame.

Add a column by assigning to a new key: a list/array must match the row count, and a **scalar broadcasts** (`df["season"] = 2022` fills every row). `df.drop(columns=[...])` returns a **new** frame and leaves the original alone unless you reassign or pass `inplace=True`.

→ THROUGH-LINE: VIEWS → COPY-ON-WRITE

On pandas 2.x, editing a filtered selection can still raise a `SettingWithCopyWarning` (the view-vs-copy hazard from Part IV). On pandas 3.0, Copy-on-Write is mandatory and that hazard is gone - a filtered subset is independent. Either way, `drop()` already returns a new frame, so a trailing `.copy()` there is redundant.

24. Filtering rows: the same mask, one library up

Filtering selects the rows that meet a condition - like a SQL `WHERE`. It is the exact two-stage mask from Part IV: build a boolean Series, then apply it.

WORKED EXAMPLE

```
bb["wins"] > 65                                # stage 1: a boolean Series
bb.loc[bb["wins"] > 65, ["city", "wins"]]      # stage 2: apply via .loc[rows, cols]
bb[bb["wins"] > 65]                            # shorthand when you don't pick columns
```

⚠ TRAP - `&` | `~`, EACH IN PARENTHESES

Combine conditions with element-wise `&` (and), `|` (or), `~` (not) - **not** Python's `and` / `or`, which raise “The truth value of a Series is ambiguous.” And wrap each condition in its own parentheses, because `&` / `|` bind tighter than `>` / `==`. This is the same rule as NumPy masks in Part IV. (Reach for `.isin([...])` or `.str.contains(...)` when they fit.)

Position vs. label: `.iloc[-1]` is the last row by position; `.loc[3]` is the row whose index *label* is 3. And remember the **attribute vs. method** tell: nouns like `.shape`, `.index`, `.loc` take no parentheses; verbs like `.info()`, `.drop()`, `.sort_values()` do. `inplace=True` mutates and returns `None`, so never write `df = df.sort_values(..., inplace=True)`.

25. NaN: the connective tissue of Module 7

Missing values show up everywhere in real data, and the same handful of facts keep paying off:

- **NaN is a float**, so the moment it lands in an integer column the whole column upcasts to `float64` (10 becomes 10.0) - one dtype per column.
- **Statistics skip NaN by default.** A row `[1, 5, NaN]` has `mean(axis=1) = 3.0` (that's 6/2, the NaN dropped - *not* 6/3).
- **EDA quantifies it:** `df.isna().sum()` counts missing per column; `df.isna().mean()` is the proportion missing (the mean of a 0/1 boolean column is its True-rate).

→ **THROUGH-LINE: DDOF=1 IS PANDAS'S DEFAULT**

`.std()` and `.sem()` use **ddof=1** (the sample statistic from Part III), while NumPy's `np.std()` defaults to `ddof=0` - same data, different number. A consequence: a column with only *one* non-NaN value returns **NaN** for `std/sem` (you cannot get a sample spread from $n=1$). And `.sem()` is $\text{std}/\sqrt{n}$ where n is the *non-null* count.

One subtlety worth burning in: the two “numeric” filters disagree on booleans. `df.mean(numeric_only=True)` includes `bool` (`True/True/True/False` averages to 0.75); `df.select_dtypes('number')` excludes `bool`. Don't treat them as interchangeable.

26. Group-by: split → apply → combine

Grouping breaks a dataset into buckets that share a key value, reduces each bucket, and stitches the answers back - **one row per group**, not one row per observation.

WORKED EXAMPLE - READ THE ONE LINE, THEN THE HIDDEN LOOP

```
df.groupby("E")["B"].mean()
#   /split by key E/ /pick B/ /reduce each bucket/

# the loop you never write:
for key in df["E"].unique():           # SPLIT into buckets
    bucket = df[df["E"] == key]
    results[key] = bucket["B"].mean() # APPLY to THIS bucket's slice only
# pandas then COMBINES into one row per key
```

KEY IDEA - EACH BUCKET SEES ONLY ITS OWN SLICE

With key `E` and data `B`: the `low` bucket `[10, 12, 7, NaN]` averages to **9.667** ($29/3$, the `NaN` skipped); the `mid` bucket `[NaN, 9, 15]` averages to **12.0**. If every bucket averaged the whole column, every output row would be identical - which is why the word to burn in is “**each**.” And `groupby` always takes the column *name*, never a value inside it (one value is a *filter*, not a *group-by*).

⚠ TRAP - NaN IS HANDLED IN TWO SEPARATE PLACES

`dropna` controls the **key** column: `dropna=True` (the silent *default*) deletes rows whose grouping value is missing - a plain `group-by` can quietly drop rows and never tell you. The `reduce` skips missing **values** on its own, regardless of `dropna`. Two different `NaN` handlers; don't conflate them.

size vs count: `.size()` counts rows in the group (`NaNs` included); `.count()` counts non-null values in the chosen column - so `size - count` is the number of missing values (in our `low` bucket, $4 - 3 = 1$). The output is sorted by *key*, not by value. And `.agg()` runs many functions × many columns in one pass - a single-row group returns `NaN` for `std/sem`, since a sample spread needs $n \geq 2$.

⚠ TRAP - A JUPYTER CELL SHOWS ONLY ITS LAST BARE EXPRESSION

End a cell with an assignment - `summary = df.groupby("E")["B"].mean()` - and you see **nothing**: no error, no table, just an empty output. Jupyter auto-displays only the last line, and only when it is a bare expression. Put the variable name on its own final line, or wrap it in `display(summary)`.

27. A real anchor, and the simulation that closes the loop

Loaded from seaborn, the Titanic data (891 rows × 15 columns) makes group-by concrete. Because `survived` is coded 0/1, the *mean is the survival rate*:

Group	passengers (count)	survival rate (mean)
1st class	216	≈ 0.63
2nd class	184 (the fewest)	≈ 0.47
3rd class	491 (the most)	≈ 0.24
overall	891	≈ 0.384

Clear stratification - survival tracked with class. (The single pooled rate of 0.384 hides large differences between classes, which is exactly what motivates breaking it down by group.) Grouping with `dropna=False` on `embark_town` even surfaces the 2 passengers whose boarding port was never recorded - a group a plain group-by would hide.

→ THROUGH-LINE: THE SEM, SIMULATED AGAIN

Module 6's capstone rebuilds Part IV's standard-error demo in pandas: generate `rg.random((25, 7))`, take `df.mean(axis=1)` (axis=1, the per-row collapse), and scatter the 25 means; then overlay a large-sample run (`rg.random((25, 500))` from the same generator). The `n=7` means scatter widely ($SD \approx 0.12$) while the `n=500` means cluster tightly around 0.5 ($SD \approx 0.014$) - the standard error falling like $1/\sqrt{n}$, now drawn in pandas. (These are 25-replication reads, so the `n=7` scatter is a noisier estimate of the same standard error §21 pinned at 0.109 with 5000 replications.)

PART V RECAP

pandas is **NumPy plus labels**: a **Series** is values + an index (no negative indexing - use `.iloc[-1]`), and a **DataFrame** is aligned Series sharing one index, where columns are variables and rows are observations. `df["c"]` → Series, `df[["a", "b"]]` → DataFrame. Filter with a boolean mask and `&` | `~` in parentheses. **NaN** is a float that upcasts columns and is skipped by stats; **group-by** splits by a column name, reduces *each* bucket's own slice, and returns one row per group - with `dropna` controlling the key separately from the reduce.

Visualization

The one big idea: you pick a plot from two facts about your variables plus one about your question. The two facts are each variable's data type (categorical or continuous) and its number of unique values; the question is whether you are showing one variable (a marginal plot) or how variables move together (a relationship plot). This is EDA's visual half - the payoff of Part V's exploration.

28. The scaffold: figure vs. axes

Three tools stack up: **matplotlib** is the low-level scaffold; **seaborn** sits on top with goal-oriented plots; **pandas** offers a quick built-in shortcut that calls matplotlib under the hood. Every matplotlib plot is four steps - define a figure and axes, plot into the axes, modify, show - and the one confusing piece is the vocabulary:

KEY IDEA - "AXES" IS NOT "AXIS"

`fig, ax = plt.subplots()` returns two objects: the **figure** (the whole canvas, can hold many plots) and the **axes** (one subplot - the plotting region). The axes is *not* the x/y axis. Ask for a grid and `axes` becomes a NumPy array you index RC (`axes[0,1]` is top-middle); `figsize=(w, h)` is in inches, width first.

29. Choosing the plot

Two properties of a variable drive the choice. **Data type:** categorical (non-numeric, either/or values; dtype `object`) vs continuous (ints/floats). **Unique count:** a numeric column with *few* unique values (rule of thumb under ~20) can be treated as categorical - in gapminder, `year` has only ~12 unique values. Then the question - one variable or a relationship - splits the rest of the module in half:

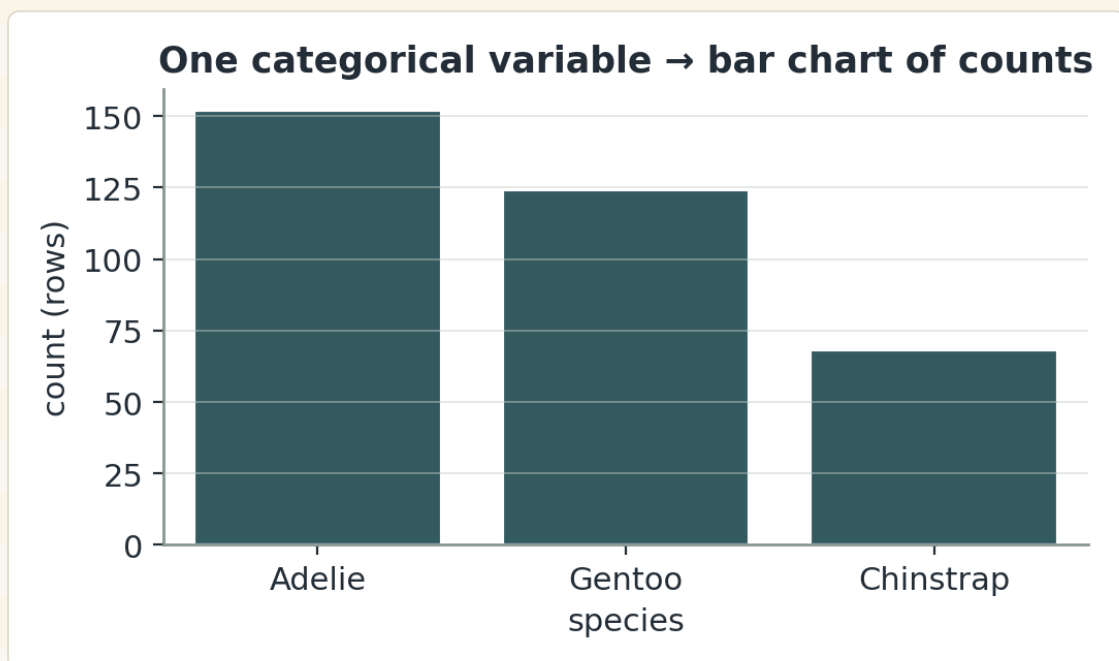
Your data	Reach for	Section
one categorical variable	bar chart of counts	\$30
one continuous variable	histogram	\$31
categorical vs continuous	boxplot	\$32
categorical vs categorical	grouped counts / heatmap / facets	\$33

→ THROUGH-LINE: EDA BECOMES PICTURES

This is Part V's exploration (counts, distributions, missing values) made visual - pipeline step 4, drawn. And the "treat a low-unique numeric column as categorical" judgment is the same instinct as reading `.dtypes` and `.nunique()` in Module 7.

30. One categorical variable → a bar chart of counts

For a single categorical variable, plot the *count* of each unique value. This is where **seaborn** enters, with two levels: **figure-level** functions (goal-oriented, easy multi-panel facets) and **axes-level** functions (finer control). For counts, `sns.catplot(data=penguins, x='species', kind='count')` is the figure-level call, and `sns.countplot(..., ax=ax)` is the axes-level function it calls underneath.

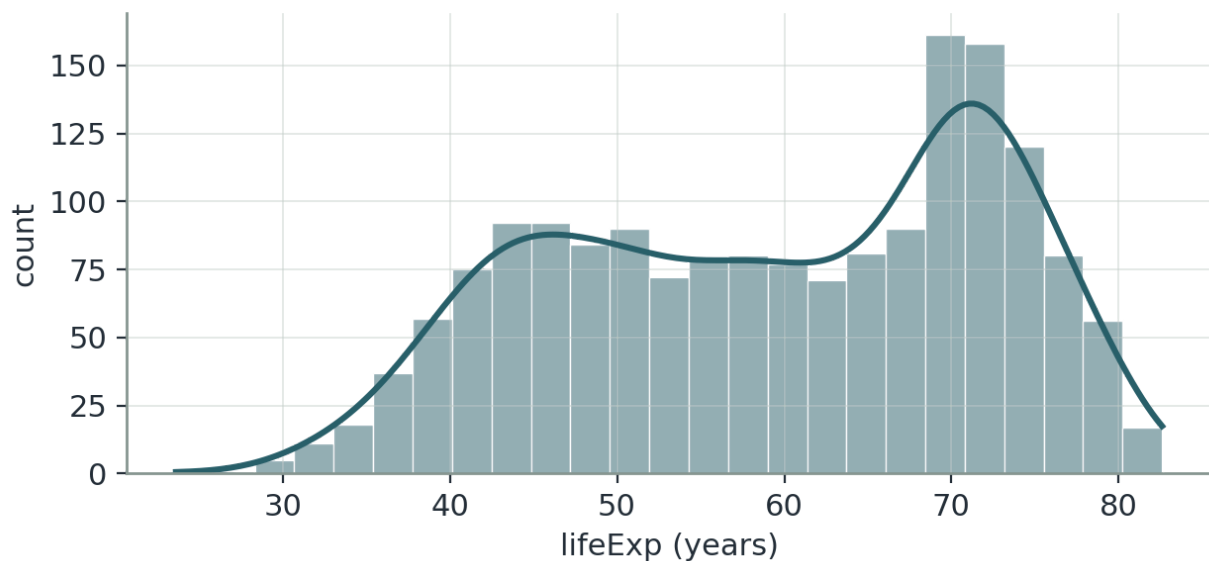


Counts of each penguin species (seaborn counts the rows for you - no manual `value_counts()` needed). Use `x=` for vertical bars, `y=` for horizontal, which reads better with long labels.

31. One continuous variable → a histogram

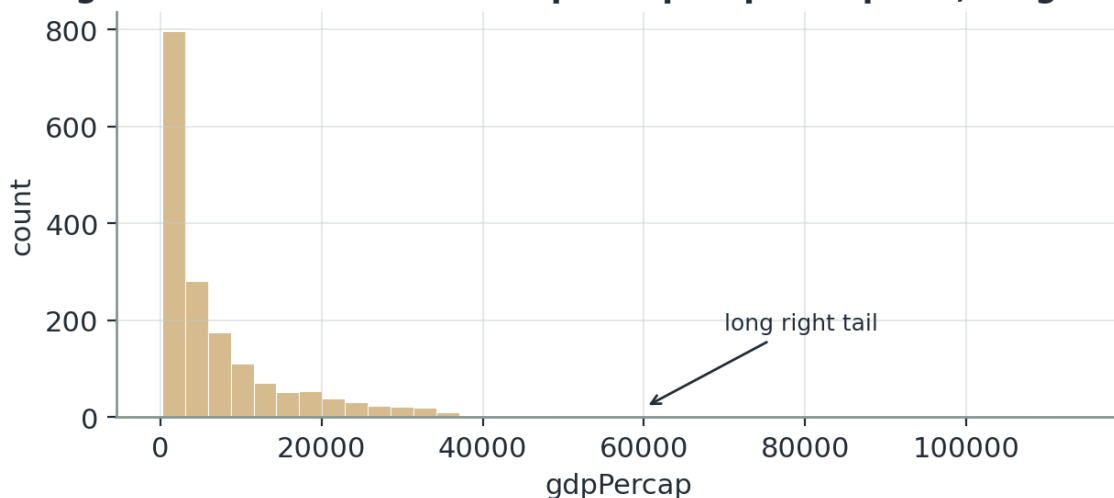
You cannot count every unique value of a continuous variable, so chop its range into **bins** and count how many values land in each. That reveals the **distribution** - where values concentrate and where they are sparse.

One continuous variable → histogram (lifeExp is bimodal)



Gapminder life expectancy is **bimodal**: a main cluster around 70–75 and a secondary group around 40–50. The smooth KDE overlay (a kernel density estimate, whose y-axis is density not counts) makes the two humps obvious. Favor a smallish number of bins - you want the shape at a glance.

A right-skewed variable: GDP per capita piles up low, long tail right



GDP per capita piles up at low values and trails off in a long right tail: it is **right-skewed** - a trait that comes back to bite boxplots in §32. (`sns.displot(kind='hist', bins=)` is figure-level; `sns.histplot` is axes-level.)

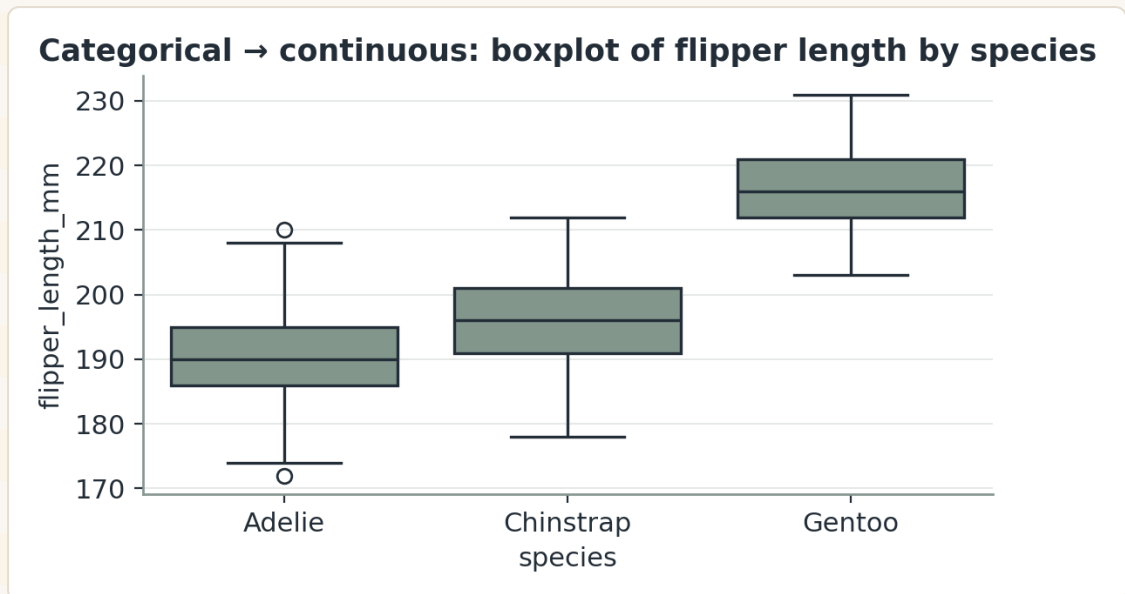
⚠ TRAP - TOO MANY BINS IS NOISE

Larger datasets can support more bins, but too many (say 200) gives choppy spikes that add noise, not information. You are after the general shape, not every wiggle. (*pandas* can do this fast too: `series.plot.hist()`, or `df.hist()` for a grid of every numeric column at once.)

32. Categorical → continuous: the boxplot

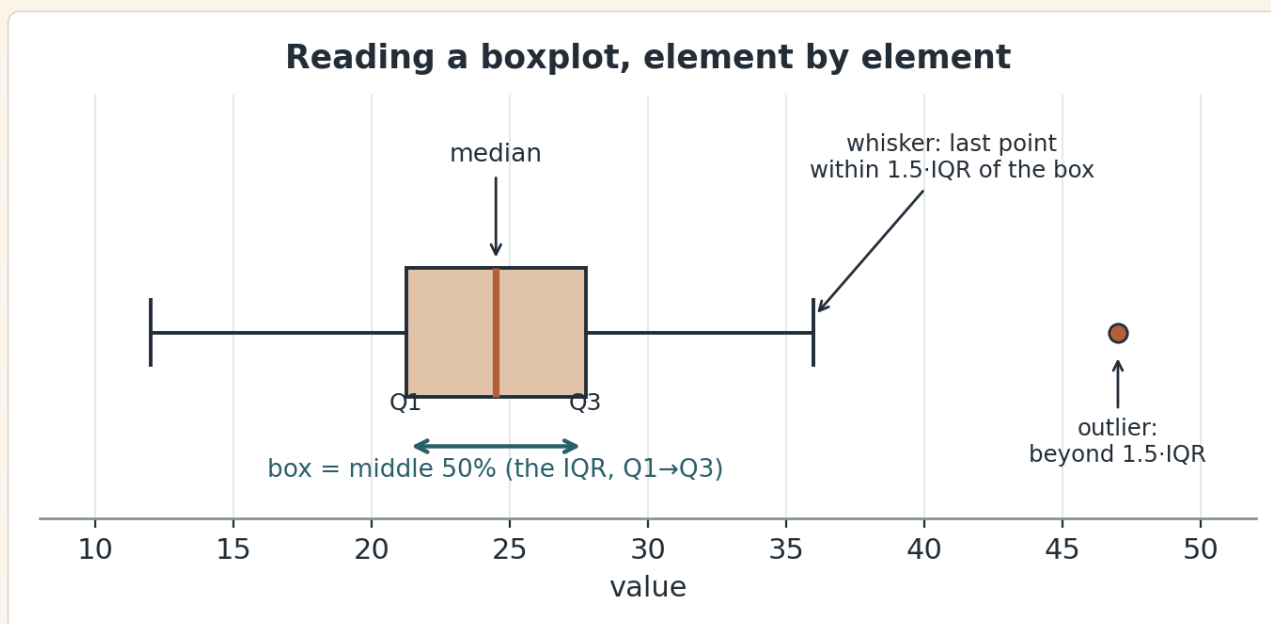
The first *relationship* plot: how does a continuous variable change across the values of a categorical one? Draw one box per category, side by side, and you can compare medians and

spreads at a glance.



Penguin flipper length by species - categorical on x , continuous on y (`sns.catplot(kind='box')`). Gentoo flippers are clearly longer; Adelie shows a couple of points flagged as outliers.

Reading a boxplot is worth doing element by element, because every mark means something specific:

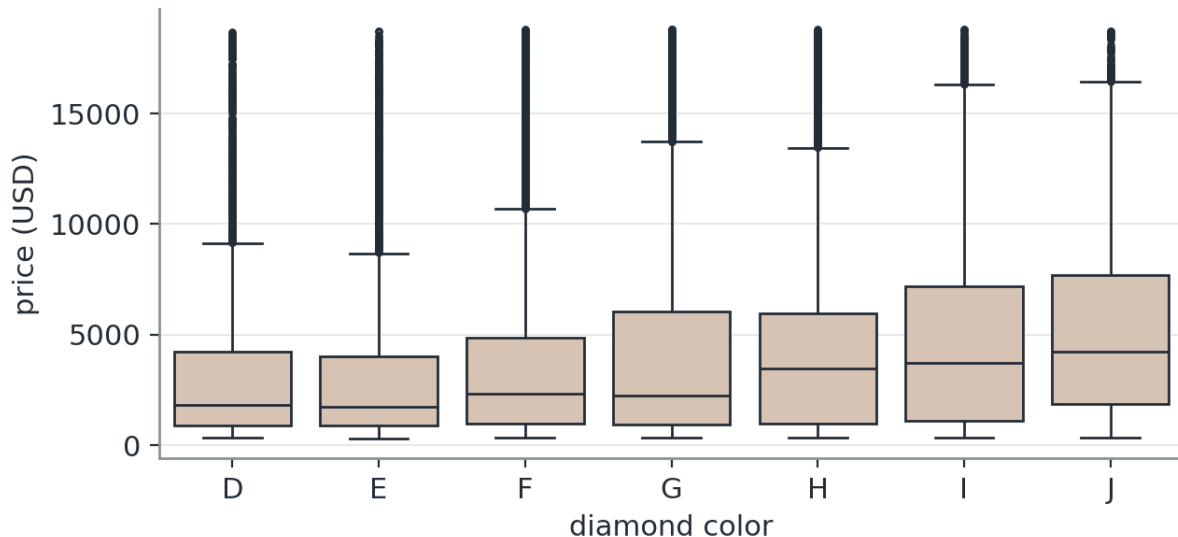


The **median** is the line inside the box (half the data above, half below - not necessarily the mean). The **box** spans $Q1$ to $Q3$, the middle 50% of the data, and its width is the **interquartile range (IQR)**. The **whiskers** reach to the most extreme point still within $1.5 \times \text{IQR}$ of the box; anything past that is plotted as an individual **outlier** dot. Any distribution works - the data does not have to be Gaussian.

⚠ TRAP - BOXPLOTS FAIL ON SKEW

Boxplots work best on roughly *symmetric* data. On a heavily skewed variable they drown in outliers and hide the real shape:

The caveat: boxplots drown in outliers on right-skewed price

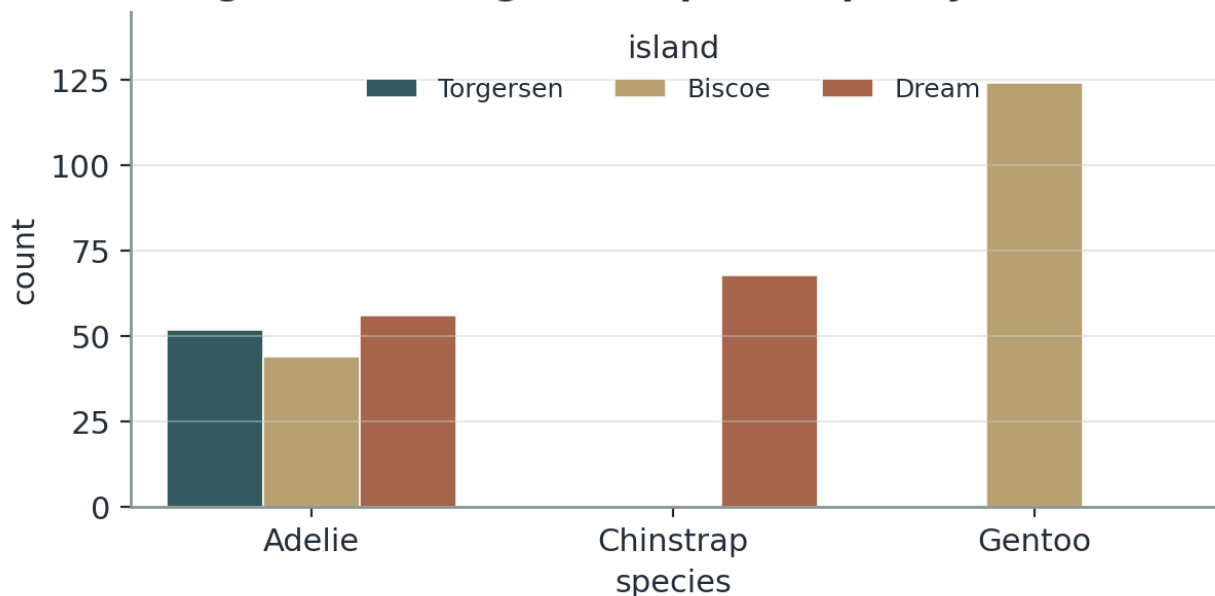


*Diamond price (strongly right-skewed) by color: each box is buried under a tower of outliers, which tells you almost nothing about the distribution's shape. When continuous data is skewed across categories, reach for **faceted histograms** or **conditional KDE plots** instead.*

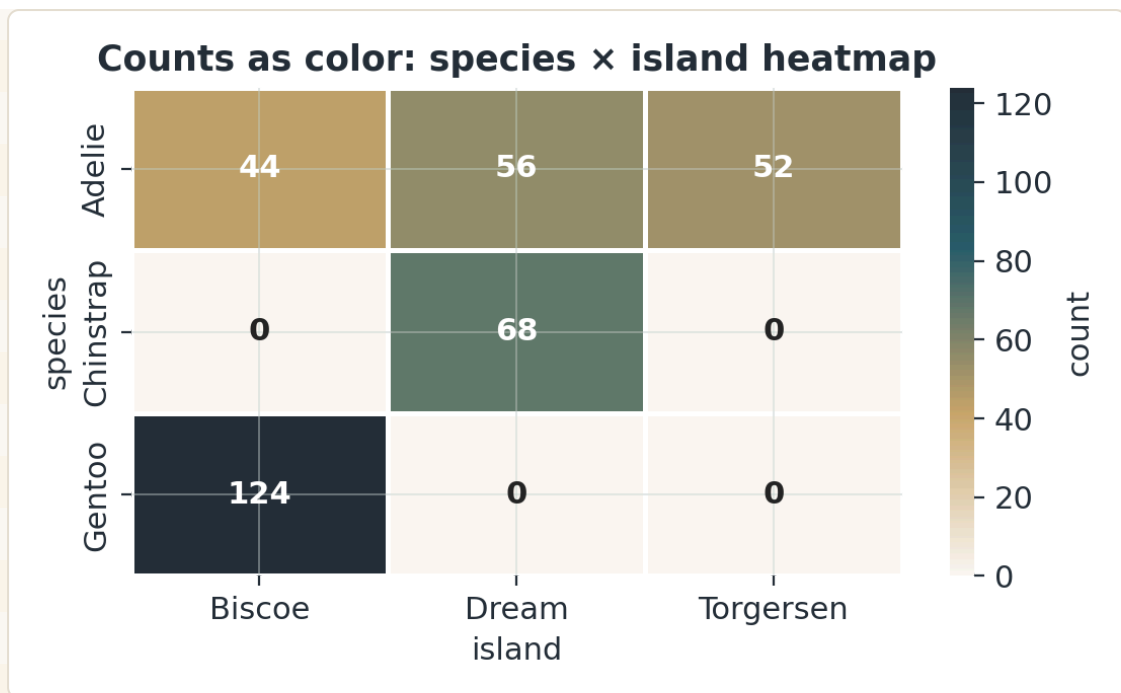
33. Categorical → categorical: grouped counts and heatmaps

The second relationship: how do the counts of one category break down across another? Add a second categorical variable to a count plot with `hue=`, a third with facets (`col=` / `row=`, which is exactly why figure-level plotting was worth setting up), or encode the counts as color in a **heatmap** built from `pd.crosstab`.

Categorical × categorical: species split by island (hue)



Penguin species split by island via `hue='island'`. Good when both variables have relatively few unique values - you can read straight off it that Chinstrap live only on Dream and Gentoo only on Biscoe.



The same two variables as a `pd.crosstab` heatmap (`sns.heatmap(ct, annot=True, fmt='d')`).
Heatmaps scale better than grouped bars when a variable has many levels; the annotated counts make the structure exact.

PART VI RECAP

The plot falls out of **data type × unique count × marginal-vs-relationship**. **matplotlib** supplies the figure/axes scaffold; **seaborn** draws goal-oriented plots (figure-level for facets, axes-level for control); **pandas** is the quick shortcut. One categorical → **bar chart**; one continuous → **histogram** (+ KDE), few bins; categorical×continuous → **boxplot** (median, box = IQR, whiskers to 1.5×IQR, outliers beyond - but it misleads on skew); categorical×categorical → **grouped counts, facets, or a heatmap**.

The Weave

Six parts, but one lesson. Here is the whole half-semester in a single breath, so you can feel the threads pull together.

It starts with a **table** (Part I): rows are observations, columns are attributes, and data science either reads patterns *across the columns* (insight) or predicts *a new row* (prediction) - a process the six-step pipeline organizes, with ~80% of the work in cleaning. To do that cleaning you write **Python** (Part II): you hold values in typed variables and lists, reach into them with zero-based, exclusive-stop slicing, repeat work with loops, and compress the most common move - transform every element of a list - into the one-line comprehension `[action for x in old]`, whose `**` operator gives you a square root as `** 0.5`.

Those two tools are exactly what the **statistics** need (Part III): variance is a comprehension over squared deviations divided by `n-1`, SD is `variance ** 0.5`, and the standard error `s/√n` shrinks like `1/√n` - the Central Limit Theorem making the bell of means normal so a confidence interval can bracket the truth. **NumPy** (Part IV) turns the table into a fast numeric grid where “axis is the direction you collapse,” and it *simulates* that `1/√n` law instead of just asserting it. **pandas** (Part V) bolts the labels back on, reuses the same axis rule and the same boolean masks, adds NaN-aware group-by, and simulates the standard error *again*. And **visualization** (Part VI) is simply the exploration step made visible - the distributions you computed, now drawn.

THE ONE IDEA

Every module is in service of a single arc: **turn a messy table into a trustworthy, well-communicated answer**. The table, the comprehension, the `1/√n` law, the collapsing axis, the mask - these are not five topics. They are five views of the same craft.

How sure we are: confidence & provenance

These notes were built with the **Double Diamond QC** process. A first diamond established the facts: an author drafted, two independent auditors hunted for errors (one tracing every claim to the source material, the other *running* every line of code and every number), and a meta-auditor reviewed them. A second diamond enriched the verified facts with examples, traces, and the graphs - each new example and figure re-run, each checked against the locked facts it illustrates - and a final full-document gate reviewed the whole thing on truth, clarity, and presentation. Every number and figure here was executed on the Anaconda stack named on the first page, not recalled.

Part	Confidence	Basis
I - Foundations	High	Concepts traced to the course's framing and pipeline material.
II - Python	Very High	Every snippet (slicing, ranges, loop traces, the comprehension, <code>**</code>) executed.
III - Statistics	High	Variance/SEM/CI numbers run; CI coverage and the CLT simulated; the interpretation trap flagged.
IV - NumPy	Very High	Array shapes, reshape/transpose, views, axis reductions, and the SEM capstone all run.
V - pandas	Very High	Filtering, NaN behavior, group-by buckets, and the Titanic numbers verified on pandas 2.3.
VI - Visualization	Very High	Every figure generated from the real datasets; crosstab counts and skew confirmed.

THE FOUR "ANSWER THE COURSE'S WAY" FLAGS

Carry these into a quiz: (1) the course calls *using an undefined variable* a syntax error (it is really a runtime `NameError`); (2) the course reads a 95% CI as "95% chance the true mean is in this interval" (really a statement about the procedure); (3) a text column's dtype is `object` on your install; (4) older lab material says an int array prints `dtype=int64` on Windows - but that is version-dependent (on your NumPy 2.x the default `int64` is hidden, since a repr only shows the dtype when it is *not* the default). Understand the precise version - answer the course's version.

CMPINF-2100 Part 1 - Canyon Notes. Authored and directed by Victor S. (DatJavaClass), with assistance by Claude. Built with the Double Diamond QC process. Some material may have been derived from assigned course material but has not been copied verbatim; for source materials please contact CMPINF-2100 Faculty and Assistants. A study aid, not a textbook - you're awesome, stay awesome, and best of grades.